V8コントリビュート超入門

西 悠太　/ 株式会社ダイニー

自己紹介

西 悠太 (Nishi Yuta)

21歳(重要、important)

V8 Contributor

Platform Engineer at Dinii Inc.

TSKaigi Staff

TypeScriptが好きです。
V8にパッチを送るのも好きです。

@riya-amemiya



今日のゴール

V8にコントリビュートする方法を覚える

V8とは？

V型8気筒（ブイがたはちきとう）は、レシプロエンジン等のシリンダー配列形式の一つで、直列4気筒2組がV字様に配置されている形式を指す。当記事では専らピストン式内燃機関のそれについて述べる注釈 1。V8（ブイはち）と略されることが多い。

多気筒レシプロエンジンとして広く用いられるエンジン形式の一つであり、自動車用としては特に大排気量車の多かったアメリカ合衆国で発達してきた。ガソリンエンジン、ディーゼルエンジン双方あるも、現代では大型乗用車用のエンジン形式として普及している。 引用: V型8気筒 - Wikipedia

V8とは？

V8は、Googleが開発するオープンソースのJIT仮想マシン型のJavaScriptエンジンである3。この名前は同じく「V8」と略されるV型8気筒エンジンに由来している4。Google ChromeなどのChromiumベースのブラウザや、Node.jsなどで採用されている。

概要
ECMAScript (ECMA-262) 準拠で、C++で記述されている。スタンドアローンでの実行が可能なほか、C++で書かれたアプリケーションの一部として動作させることもできる。

引用: V8 (JavaScriptエンジン) - Wikipedia

リポジトリはGitHubではありません

```
https://chromium.googlesource.com/v8/v8.git   本体
https://github.com/v8/v8                      公式ミラー
```

GitHubで検索すると `v8/v8` が出てきますが、あれは公式ミラーです。

ミラーにPull Requestは送れません

→ GitHub側の Issues は開いていません

→ Pull Request も受け付けていません

V8のコードレビューは、Chromiumと同じGerritで行われます。

Gerrit

chromium-review.googlesource.com

言葉の対応

Pull Request	→	**CL (Change List)**
Merge ボタン	→	**Commit Queue**
CI	→	**try bot**

全体の流れ

1. depot_tools を入れる
2. ソースを取る
3. ビルドする
4. 直して、テストを足す
5. CLを送る
6. レビューを受けて、Commit Queueに載せる

depot_tools

Chromium と V8 の開発には depot_tools が必須です。

git clone して、パスを通します。

```
git clone https://chromium.googlesource.com/chromium/tools/depot_tools.git
export PATH="$(pwd)/depot_tools:$PATH"
```

1. ソースを取る

```
fetch v8
cd v8
```

`fetch` は depot_tools に入っています。

取得のあと、意図的に detached HEAD になります。

Gerritの設定&mainにswitchします。

```
git config --local gerrit.host true
git switch main # or checkout main
```

2. 依存取得&ビルド

```
git pull origin main
gclient sync
```

```
./tools/dev/gm.py arm64.release
```

`gm.py` でビルドします。
Macbook ProのM4 Proで1時間ぐらいかかります

3. 直して、テストを足す

初めてのときは、AUTHORS に名前とメールを追記します。最初のパッチに含めます。

変更にはテストが必要です。

- → JavaScriptから見える変更は `test/mjsunit/`
- → C++の単体テストは `test/unittests/`
- → バグ修正の回帰テストは `test/mjsunit/regress/`

回帰テストのファイル名

```
test/mjsunit/regress/regress-crbug-1000094.js
```

`regress-crbug-` のあとにバグ番号を付けます。

4. CLを送る

まずブランチを作ります。

```
git new-branch mychange
# ~コードを編集したり、テストを書いたりする~
```

`git checkout -b` でも問題ありませんが、公式では `git new-branch` が推奨されています。

コミットメッセージ

[component] Fix various typos in comments

Bug: v8:12345

Gerritのアカウント

chromium-review.googlesource.com に、コミットに使う Google アカウントでログインします。

コミットのメールアドレスと、Google アカウントのメールアドレスを一致させます。

初回だけ、CLAを出します

```
https://cla.developers.google.com/
```

GoogleのContributor License Agreementに署名します。

Uploadする

```
git cl format
git cl presubmit
git cl upload
```

`git cl upload` を打つと、Gerrit 上に CL ができます。

5. レビューとCommit Queue

```
git cl owners
```

- → 変更したディレクトリの OWNERS から承認をもらいます
- → Commit-Queue +1 は、コミットせずに try bot だけ回します
- → Commit-Queue +2 で、testが全て成功したら自動でコミットされます
- → (CQを押す権限はcommitterにのみあります)

Commit-Queue +2されてから
約30分待ったらマージされます🎉

今日覚えて帰ってほしいこと

- ✓ V8の本体はchromium.googlesource.comにあり、GitHubの `v8/v8` はミラー
- ✓ 変更は、GerritにCLとして送る
- ✓ depot_tools を入れてから `fetch v8` でソースを取り、 `tools/dev/gm.py` でビルドする
- ✓ 挙動が変わる変更には、テストを付ける
- ✓ OWNERSの承認が付いてCommit Queueを通ると、自動でコミットされる

参考資料

- V8 Docs: **Contributing**
- depot_tools: **Setting up**
- V8 Docs: **Checking out the V8 source code**
- V8 Docs: **Building V8 from source**
- Chromium Docs: **Contributing to Chromium**

ご清聴ありがとうございました